



Efficient SQL in Yellowbrick

Module 5: Efficient SQL in Yellowbrick

Purpose

Learn to write efficient, readable SQL against Yellowbrick — from your very first SELECT to window functions — with practical MPP tips woven in throughout.

Main Topics

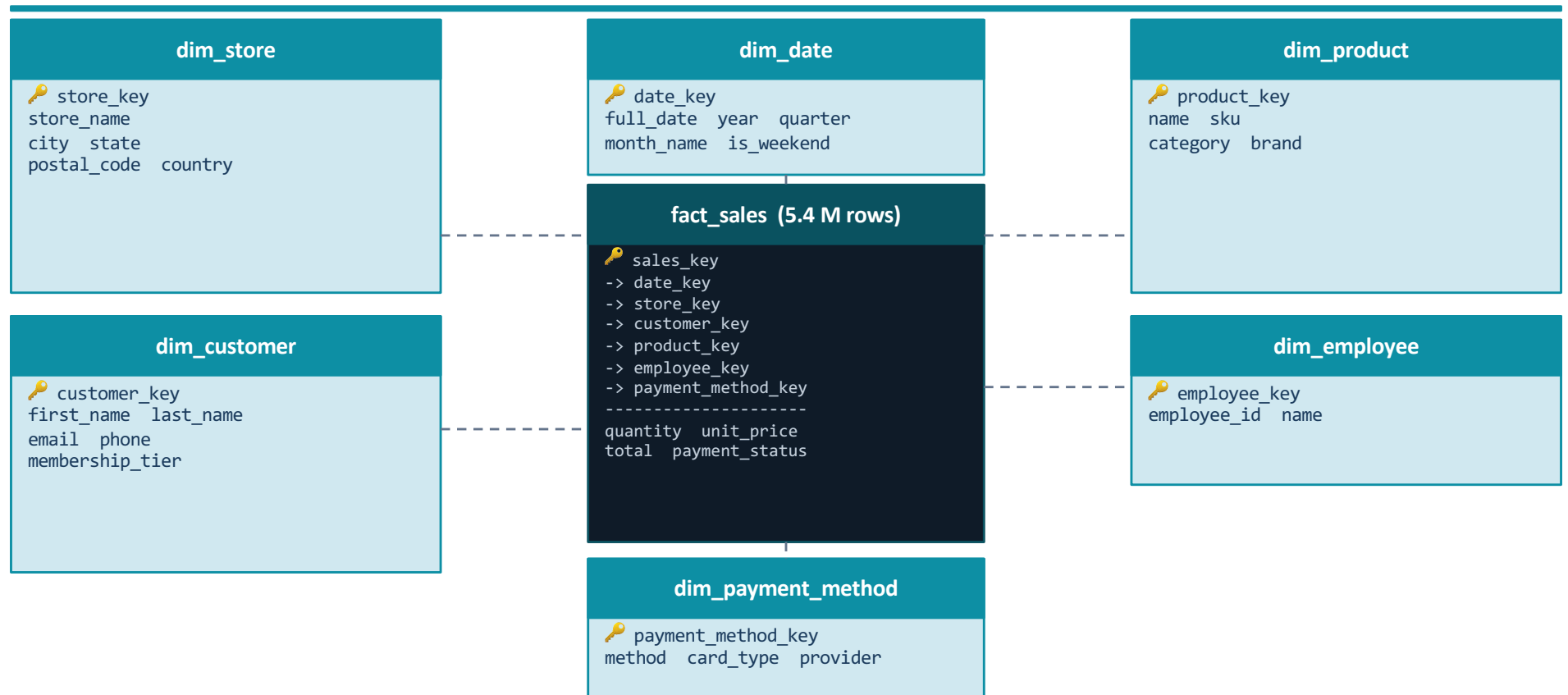
SELECT basics · Filtering (WHERE) · Sorting (ORDER BY) · Aggregating (GROUP BY, COUNT, SUM) · JOINS · Window functions · CTEs · Efficiency tips

Audience

Business users and analysts — no SQL experience required. If you can read a spreadsheet you have everything you need.

 *All examples use the `samples.retail` schema — a retail data warehouse you already have read access to.*

Our Dataset: samples.retail — A Retail Star Schema



🔑 = primary key -> = foreign key



Section 1 · What Is SQL?

The language that talks to databases — and why Yellowbrick speaks it natively

What Is SQL?

SQL (Structured Query Language) is a simple but powerful language for asking questions of a database.

You describe

what you want — not **how** to get it. Yellowbrick figures out the most efficient path.

```
-- Ask: which stores are in Texas?  
SELECT store_name, city  
FROM   samples.retail.dim_store  
WHERE  state = 'TX';
```



Query

Ask a question — "show me all sales over \$500 this month"



Transform

Calculate, group, or reshape data on the way out



Report

Feed results directly to BI tools, dashboards, or exports



Section 2 · Your First SELECT

SELECT, FROM, and the semicolon — the three things every SQL query needs

Anatomy of a SQL Query

```
SELECT store_name,  
       city,  
       state  
FROM   samples.retail.dim_store  
LIMIT 10;
```

SELECT Which columns do you want to see?

FROM Which table are you reading from? Format:
database.schema.table

LIMIT Return at most N rows — great for previewing
large tables

; Every statement ends with a semicolon

SELECT * vs. Specific Columns

❌ SELECT * — reads everything

```
-- Reads ALL 27 columns from fact_sales
-- Yellowbrick still transfers every
-- column across the network
SELECT *
FROM   samples.retail.fact_sales
LIMIT 10;
```

Named columns — reads only what you need

```
-- Only fetches 4 columns
-- Much less I/O on a wide table
SELECT transaction_id,
       total,
       payment_status,
       transaction_timestamp
FROM   samples.retail.fact_sales
LIMIT 10;
```

⚡ **Columnar storage makes this matter** Yellowbrick stores each column separately on disk. Selecting 4 of 27 columns means it physically skips the other 23 — less disk I/O = faster queries, every time.

Giving Columns Friendly Names with AS

Column names from the database are often cryptic codes. AS lets you rename them for clarity in your results and downstream tools.

```
SELECT transaction_id      AS "Transaction",
       store_key          AS "Store ID",
       quantity           AS "Qty",
       unit_price         AS "Unit Price",
       quantity * unit_price AS "Sale Total",
       transaction_timestamp AS "Date & Time"
FROM   samples.retail.fact_sales
LIMIT 5;
```

AS is optional but recommended

AS "My Name" is clearest; some tools need quotes for spaces.

Aliases are for output only

You cannot use the alias in the WHERE clause of the same query.

Use in GROUP BY and ORDER BY

SELECT total AS sale_total ... ORDER BY sale_total is valid.

Expressions too

SELECT unit_price * quantity AS line_total is perfectly valid.



Section 3 · Filtering with WHERE

Tell Yellowbrick exactly which rows you want — and get nothing more

WHERE: Basic Comparisons

Op	Meaning	Example
=	Equals	state = 'TX'
<>	Not equal	payment_status <> 'refunded'
>	Greater than	total > 1000
<	Less than	unit_price < 50
>=	Greater or equal	quantity >= 3
<=	Less or equal	tax_rate <= 0.10

```
-- Stores in Texas
SELECT store_name, city
FROM   samples.retail.dim_store
WHERE  state = 'TX';

-- Sales over $2,000
SELECT transaction_id, total
FROM   samples.retail.fact_sales
WHERE  total > 2000
LIMIT 5;
```

Combining Conditions: AND, OR, NOT

Combine multiple conditions to narrow or broaden your results. Use parentheses when mixing AND and OR — they change the meaning.

```
-- AND: both conditions must be true
SELECT store_name, city, state
FROM   samples.retail.dim_store
WHERE  state = 'TX'
      AND city = 'Fairview';

-- OR: either condition is enough
SELECT store_name, city, state
FROM   samples.retail.dim_store
WHERE  state = 'CA'
      OR state = 'NY';

-- NOT: exclude matching rows
SELECT transaction_id, payment_status
FROM   samples.retail.fact_sales
WHERE  NOT payment_status = 'completed'
LIMIT 5;
```

Operator Precedence

AND binds tighter than OR.

```
state = 'CA' OR state = 'NY' AND total > 1000
```

...means: CA (any total) OR (NY AND total>1000).

Always use parentheses when mixing:

```
(state = 'CA' OR state = 'NY') AND total > 1000
```

 **When in doubt, add parentheses** They cost nothing and prevent hard-to-spot bugs when conditions are re-ordered.

BETWEEN, IN, and ILIKE

BETWEEN — range check (inclusive)

```
-- Transactions between $100 and $500
SELECT transaction_id, total
FROM   samples.retail.fact_sales
WHERE  total BETWEEN 100 AND 500
LIMIT 5;
```

IN — match any value in a list

```
-- Stores in CA, NY, or TX
SELECT store_name, state
FROM   samples.retail.dim_store
WHERE  state IN ('CA', 'NY', 'TX');
```

ILIKE — pattern match (case-insensitive)

```
-- Products whose name contains 'pro'
SELECT name, category, brand
FROM   samples.retail.dim_product
WHERE  name ILIKE '%pro%'
LIMIT 5;
-- % matches any sequence of characters
```

⚡ **Use ILIKE in Yellowbrick** Standard SQL uses LIKE (case-sensitive). Yellowbrick supports ILIKE for case-insensitive matching — much friendlier for name or description searches.

Handling NULL Values

NULL means "no value recorded" — it is not zero, not empty string, and it is not equal to anything — including itself. You must use special operators to test for it.

```
-- ✅ Correct: test with IS NULL / IS NOT NULL
SELECT transaction_id, discount_type, total
FROM   samples.retail.fact_sales
WHERE  discount_type IS NOT NULL
LIMIT 5;

-- ❌ Wrong: = NULL never matches anything
WHERE discount_type = NULL -- always returns 0 rows

-- NULLs in ORDER BY go last by default
SELECT name, category
FROM   samples.retail.dim_product
ORDER BY category ASC NULLS LAST
LIMIT 5;
```

NULL in arithmetic

Any calculation involving NULL returns NULL.
 $5 + \text{NULL} = \text{NULL}$

NULL in aggregates

COUNT(*) counts every row.
COUNT(col) skips NULLs.

COALESCE()

Replace NULL with a default:
COALESCE(discount_type, 'none')

YB Tip: Filter Smart — How WHERE Helps Yellowbrick Skip Work

Yellowbrick stores rows in ordered blocks on disk. When your filter matches the table's sort order, Yellowbrick can skip entire blocks it doesn't need to read — called a block scan skip.

❌ Filter on an expression — kills scan optimization

```
-- Yellowbrick cannot skip blocks because
-- YEAR() wraps every value before comparing
SELECT *
FROM   samples.retail.fact_sales
WHERE  YEAR(transaction_timestamp) = 2026;
```

✅ Direct range filter — Yellowbrick can skip blocks

```
-- Yellowbrick reads the range directly
-- and skips blocks outside it
SELECT *
FROM   samples.retail.fact_sales
WHERE  transaction_timestamp
      >= '2026-01-01'
      AND transaction_timestamp
      <  '2027-01-01';
```

⚡ **Rule of thumb** Avoid wrapping filter columns in functions (YEAR(), LOWER(), TRIM(), etc.). Filter on the raw column value whenever possible.



Section 4 · Sorting with ORDER BY

Control which rows come back first — and how many

ORDER BY and LIMIT

```
-- Sort stores alphabetically by state, then city
SELECT store_name, city, state
FROM   samples.retail.dim_store
ORDER  BY state ASC, city ASC
LIMIT  8;

-- Top 5 highest-value transactions
SELECT transaction_id,
       total,
       transaction_timestamp
FROM   samples.retail.fact_sales
ORDER  BY total DESC
LIMIT  5;

-- ASC = smallest first (default)
-- DESC = largest first
```

Multiple sort columns

List them in priority order. First column breaks ties with the second.

ASC is the default

You can omit it, but writing it makes the intent explicit.

LIMIT is your best friend

Always LIMIT when exploring a large table. It stops Yellowbrick from returning millions of rows you don't need to see.

LIMIT without ORDER BY

Returns any N rows — Yellowbrick chooses. Add ORDER BY when row order matters.

⚡ **ORDER BY and sort keys** Yellowbrick tables are stored in sorted order (like an index). Sorting by that same column is nearly instant — no extra work needed.



Section 5 · Aggregating Data

Turn millions of rows into the numbers you actually care about

Aggregate Functions: COUNT, SUM, AVG, MIN, MAX

```
-- Single query, five aggregate functions
SELECT COUNT(*)           AS total_transactions,
       ROUND(SUM(total), 2) AS total_revenue,
       ROUND(AVG(total), 2) AS avg_sale,
       ROUND(MIN(total), 2) AS smallest_sale,
       ROUND(MAX(total), 2) AS largest_sale
FROM   samples.retail.fact_sales;

-- Result (actual data):
-- total_transactions: 5,493,074
-- total_revenue:      8,701,956,042.14
-- avg_sale:           1,584.16
-- smallest_sale:       0.54
-- largest_sale:        5,437.39
```

COUNT(*)

Counts every row, including those with NULLs

COUNT(col)

Counts only rows where col is NOT NULL

SUM(col)

Adds up all values — works on numbers only

AVG(col)

Arithmetic mean — ignores NULLs

MIN(col)

Smallest value — works on dates too

MAX(col)

Largest value — works on dates too

ROUND(x,n)

Round to n decimal places (not an aggregate, but useful here)

GROUP BY: Aggregating by Category

GROUP BY splits your data into groups and applies an aggregate function to each group independently. Every column in SELECT must either be in GROUP BY or wrapped in an aggregate function.

```
-- Revenue by product category
SELECT p.category,
       COUNT(*)           AS transactions,
       ROUND(SUM(f.total), 0) AS revenue
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_product p
       ON f.product_key = p.product_key
GROUP BY p.category
ORDER BY revenue DESC
LIMIT 6;

-- Top result:
-- category: Furniture | revenue: 447,052,019
```

The GROUP BY rule

In SELECT

Every column must be either:

1. Listed in GROUP BY, OR
2. Inside an aggregate (COUNT, SUM...)

In GROUP BY

List every non-aggregate column from SELECT.

Tip

You can GROUP BY column number:

GROUP BY 1, 2
matches the 1st and 2nd SELECT columns.

HAVING: Filtering on Aggregated Results

WHERE filters individual rows before grouping. **HAVING** filters groups after aggregation.

```
-- Only show categories with revenue > $430 million
SELECT p.category,
       ROUND(SUM(f.total), 0) AS revenue
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_product p
       ON f.product_key = p.product_key
GROUP  BY p.category
HAVING SUM(f.total) > 430000000 -- filter on the group
ORDER  BY revenue DESC;

-- Result: Furniture, Clothing, Electronics,
-- Tools & Hardware, Computers, Books (6 rows)
```

SQL Execution Order

- | | | |
|---|----------|--------------------------|
| 1 | FROM | Which table(s)? |
| 2 | WHERE | Filter individual rows |
| 3 | GROUP BY | Form groups |
| 4 | HAVING | Filter groups |
| 5 | SELECT | Choose columns / compute |
| 6 | ORDER BY | Sort result |
| 7 | LIMIT | Trim to N rows |

YB Tip: GROUP BY and Data Distribution

Yellowbrick distributes table rows across many compute nodes. When you GROUP BY, each node must send its local groups to a single node for final merging — unless the data is already grouped where it lives.

✓ GROUP BY the distribution column

```
-- fact_sales is distributed on customer_key
-- Each node groups its own customers locally
-- No cross-node data movement needed
SELECT customer_key,
       COUNT(*) AS txns
FROM   samples.retail.fact_sales
GROUP BY customer_key
ORDER BY txns DESC LIMIT 5;
```

⚠ GROUP BY many low-cardinality columns

```
-- Low cardinality = few distinct combos
-- All data funnels to a handful of nodes
-- (e.g. 20 categories × 12 months = 240 groups)
SELECT p.category,
       d.month_name,
       SUM(f.total) AS revenue
FROM   ...
GROUP BY p.category, d.month_name;
```

⚡ **Ask your DBA what columns tables are distributed on** Joining or grouping on a distribution column is always fastest. It is usually the primary key or the most commonly joined foreign key.



Section 6 · Joining Tables

Pull related information from multiple tables into a single result

What Is a Join?

A join combines rows from two tables wherever a column value matches — like a VLOOKUP in Excel, but for entire datasets at once.



⚡ **Always give tables short aliases** JOIN samples.retail.fact_sales f — then use f.total instead of the full path. Keeps queries readable.

INNER JOIN: Only Matching Rows

INNER JOIN returns only rows where the join condition is met in BOTH tables. If a row in one table has no match in the other, it is excluded.

```
-- Enrich each sale with the store name
SELECT f.transaction_id,
       f.total,
       s.store_name,
       s.city,
       s.state
FROM   samples.retail.fact_sales  f
JOIN   samples.retail.dim_store  s
      ON f.store_key = s.store_key
LIMIT 5;

-- JOINS a 5.4M-row fact table to a dim
-- Only sales that have a matching store appear
```

JOIN = INNER JOIN

The word INNER is optional. Most people just write JOIN.

ON clause

Specifies which columns to match: ON f.store_key = s.store_key

Table aliases

f and s shorten the table names. Use them throughout the query.

Multiple JOINS

You can chain as many JOINS as you need — one per dimension table.

LEFT JOIN: All Rows from the Left Table

LEFT JOIN keeps every row from the first (left) table, even if there is no match in the second table. Unmatched columns from the right table will be NULL.

```
-- All customers, plus their purchase totals
-- Customers with no purchases still appear (NULL totals)
SELECT c.first_name,
       c.last_name,
       c.membership_tier,
       COUNT(f.sales_key)      AS purchases,
       ROUND(SUM(f.total), 0)  AS total_spent
FROM   samples.retail.dim_customer c
LEFT   JOIN samples.retail.fact_sales f
       ON c.customer_key = f.customer_key
GROUP  BY c.first_name, c.last_name, c.membership_tier
ORDER  BY total_spent DESC NULLS LAST
LIMIT  5;
```

INNER JOIN vs LEFT JOIN

INNER JOIN

Returns matched rows only.
Customers with no purchases: excluded.

LEFT JOIN

Returns ALL left rows.
Customers with no purchases: included, purchases = NULL.

When to use LEFT JOIN

When you want to see what is missing — "which customers have never bought?" WHERE f.sales_key IS NULL

Cross Join: Every Row Meets Every Row

A CROSS JOIN produces the Cartesian product of two tables — every row in the first table paired with every row in the second. No ON clause is used.

```
-- Intentional cross join:
-- every store paired with every product
SELECT s.store_name,
       p.name      AS product,
       p.category
FROM   samples.retail.dim_store s
CROSS JOIN samples.retail.dim_product p
ORDER BY s.store_name, p.category
LIMIT 10;

-- Row count math:
-- dim_store: 100 rows
-- dim_product: ~50,000 rows
-- Result: 100 x 50,000 = 5,000,000 rows
```

What a CROSS JOIN produces

The math

Row count = rowsA x rowsB, always.
100 stores x 50,000 products = 5,000,000 rows.
The database will not warn you.

Syntax

```
FROM table_a CROSS JOIN table_b
```

No ON clause. No WHERE needed to trigger it.
Every combination is returned.

When it is intentional

Generating a full grid: every store x every product.
Filling calendar gaps: every date x every region.
Building a reference matrix for reporting.

The Accidental Cross Join: A Silent Row Explosion

The most dangerous cross join is the one you did not mean to write. A JOIN missing its ON clause silently multiplies your rows — no error, just wrong answers.

```
-- MISSING ON CLAUSE
-- This is a cross join in disguise!
SELECT f.transaction_id,
       s.store_name,
       f.total
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_store s;
-- ^ No ON clause!

-- fact_sales: 5.4M rows
-- dim_store: 100 rows
-- Result: 540,000,000 rows
-- No error -- just silently wrong

-- SUM(f.total) will be 100x too large
```

Spot it • Fix it

How to notice it

Row count = rowsA x rowsB, not what you expect. Aggregates like SUM() return values that are a round multiple of the other table's row count. Results feel too large and suspiciously round.

How to fix it

Add the correct ON clause:

```
JOIN   samples.retail.dim_store s
      ON f.store_key = s.store_key
```

Quick check

Run SELECT COUNT(*) on just the join alone, before adding any WHERE filters. If count > max(rowsA, rowsB), you have a cross join somewhere.

Joining Multiple Tables

Chain as many JOINS as you need — one per dimension table. Each JOIN adds columns from that dimension to your result set.

```
SELECT f.transaction_id,  
       f.total,  
       s.store_name,  
       s.state,  
       p.name      AS product,  
       p.category,  
       c.first_name || ' ' || c.last_name AS customer,  
       c.membership_tier,  
       d.full_date  
FROM   samples.retail.fact_sales      f  
JOIN   samples.retail.dim_store      s ON f.store_key    = s.store_key  
JOIN   samples.retail.dim_product    p ON f.product_key  = p.product_key  
JOIN   samples.retail.dim_customer   c ON f.customer_key = c.customer_key  
JOIN   samples.retail.dim_date       d ON f.date_key     = d.date_key  
LIMIT 5;
```

⚡ **Qualify column names when joining multiple tables** Once you join two tables, identical column names (like `store_key`) become ambiguous. Always prefix with the alias: `f.store_key`, `s.store_key`.

YB Tip: How Yellowbrick Executes Joins

In an MPP database, rows are spread across many nodes. To join two tables, Yellowbrick must get matching rows onto the same node — that data movement is the most expensive part of a join.



Best: No data movement

Both tables are distributed on the join column. Each node handles its own rows locally — instant.



Good: Redistribute the small side

Large table stays put; the small dimension table is broadcast to each node. Most fact-to-dim joins fall here.



Slower: Both sides redistributed

Neither table is on the join column — both sides must shuffle. Avoid if possible by joining on distribution keys.

⚡ Dimension tables are usually small enough to broadcast For the retail schema, dim_store (100 rows) or dim_product (~50k rows) will always be broadcast. Don't over-optimize joins to small lookup tables.



Section 7 · Window Functions

Calculations that look across rows — without collapsing them into groups

What Is a Window Function?

A window function computes a value for each row using a related set of rows — its "window" — without collapsing the result into one row the way GROUP BY does.

	GROUP BY + SUM	SUM() OVER (window)
Output rows	One row per group	One row per input row (preserved)
Use case	Total revenue per category	Show each sale + running total alongside it
Collapses data?	Yes	No

```
-- Syntax: function() OVER (partition + order)
SELECT transaction_id,
       total,
       SUM(total) OVER (ORDER BY transaction_timestamp
                        ROWS UNBOUNDED PRECEDING) AS running_total
FROM   samples.retail.fact_sales
LIMIT 5;
```


RANK() and ROW_NUMBER(): Ranking Within Groups

```
-- Top 3 products by revenue within each category
SELECT p.category,
       p.name           AS product,
       ROUND(SUM(f.total), 0) AS revenue,
       RANK() OVER (
         PARTITION BY p.category
         ORDER BY SUM(f.total) DESC
       )                AS rank_in_cat
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_product p
      ON f.product_key = p.product_key
GROUP BY p.category, p.name
ORDER BY p.category, rank_in_cat
LIMIT 6;
```

Choose the right ranking function

ROW_NUMBER()

Sequential 1, 2, 3... No ties ever. Two identical revenues still get different numbers.

RANK()

Skips after ties: 1, 2, 2, 4. Use when you want "tied for 2nd" to show correctly.

DENSE_RANK()

No skips: 1, 2, 2, 3. Use when you want no gaps after ties.

⚡ **PARTITION BY is optional** Omit it and the ranking applies across the entire result set. Include it to rank within sub-groups (like per category here).

Running Totals with SUM() OVER

A running total accumulates a value row-by-row in order. This is a classic use case for window functions — impossible to express cleanly with GROUP BY.

```
-- Daily revenue with a cumulative running total
SELECT d.full_date,
       ROUND(SUM(f.total), 0)           AS daily_revenue,
       ROUND(
         SUM(SUM(f.total)) OVER (
           ORDER BY d.full_date
           ROWS UNBOUNDED PRECEDING
         ), 0
       )                               AS running_total
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_date   d
      ON f.date_key = d.date_key
GROUP BY d.full_date
ORDER BY d.full_date
LIMIT  8;

-- Sample result:
-- 2025-05-15 | 8,140,736 | 8,140,736
-- 2025-05-16 | 22,646,468 | 30,787,204
-- 2025-05-17 | 24,742,191 | 55,529,395
```

The OVER() clause explained

PARTITION BY col

Restart the window for each new value of col (optional)

ORDER BY col

Defines the row sequence within the window

ROWS UNBOUNDED PRECEDING

Include all prior rows up to current row — this is what makes it a running total

SUM(SUM(...))

The outer SUM is the window function; the inner SUM is the GROUP BY aggregate

LAG() and LEAD(): Comparing Across Rows

LAG() looks back at the previous row's value; LEAD() looks ahead. Both are perfect for period-over-period comparisons — no self-join needed.

```
-- Daily revenue vs. prior day, and % change
SELECT
  d.full_date,
  ROUND(SUM(f.total), 0)                AS daily_revenue,
  ROUND(LAG(SUM(f.total)) OVER
    (ORDER BY d.full_date), 0)          AS prev_day,
  ROUND(
    (SUM(f.total)
     - LAG(SUM(f.total)) OVER (ORDER BY d.full_date))
    / LAG(SUM(f.total)) OVER (ORDER BY d.full_date)
    * 100, 1
  )                                     AS pct_change
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_date   d ON f.date_key = d.date_key
GROUP BY d.full_date
ORDER BY d.full_date
LIMIT 5;
-- 2025-05-16 | 22,646,468 | 8,140,736 | +178.2%
```

LAG(expr, n, default)

Gets the value from n rows before the current row. n defaults to 1. default fills the first row where there is no prior row.

LEAD(expr, n, default)

Gets the value from n rows ahead. Use for forecasting — "next month's target vs. this month's actuals."

Common pattern

Wrap in a CTE first (see next section) to avoid repeating the LAG expression in the pct_change calculation.

YB Tip: Window Functions in an MPP Database

Window functions are fully parallelized in Yellowbrick — each PARTITION BY group is processed independently across nodes. A few patterns need extra care.

✓ Good: PARTITION BY a high-cardinality column

PARTITION BY customer_key spreads work evenly across nodes — one partition per customer. Fast and parallel.

```
RANK() OVER (PARTITION BY customer_key
             ORDER BY total DESC)
```

⚠ Watch out: No PARTITION BY (global window)

SUM() OVER (ORDER BY date) with no PARTITION BY processes ALL rows on a single node. Fine for small results, slow on millions of rows.

```
SUM(total) OVER (ORDER BY full_date
                 ROWS UNBOUNDED PRECEDING)
```

⚠ Watch out: PARTITION BY a low-cardinality column

PARTITION BY category (only 20 values) means 20 nodes do the work while the rest idle. Skew.

```
RANK() OVER (PARTITION BY category
             ORDER BY total DESC)
```



Section 8 · Writing Cleaner SQL

Habits that make your queries easier to read, maintain, and debug



Sub-Selects: Queries Inside Queries

Use a SELECT result as a value, a list, or a table -- without a CTE

Sub-Selects in WHERE: IN and EXISTS

A sub-select inside a WHERE clause lets you filter based on the results of another query -- no extra JOIN required.

```
-- IN: customers who bought Electronics
SELECT c.first_name, c.last_name,
       c.membership_tier
FROM   samples.retail.dim_customer c
WHERE  c.customer_key IN (
    SELECT f.customer_key
    FROM   samples.retail.fact_sales f
    JOIN   samples.retail.dim_product p
        ON f.product_key = p.product_key
    WHERE  p.category = 'Electronics'
);

-- EXISTS: stores that had a sale over $5,000
SELECT s.store_name, s.state
FROM   samples.retail.dim_store s
WHERE  EXISTS (
    SELECT 1
    FROM   samples.retail.fact_sales f
    WHERE  f.store_key = s.store_key
    AND    f.total > 5000
);
```

IN vs EXISTS

IN

Matches rows where a column equals any value returned by the sub-select.
Best when the sub-select returns a small list.

EXISTS

Returns a row if the sub-select finds at least one match. Stops searching immediately once a match is found -- often faster on large tables.

NOT IN vs NOT EXISTS

Use NOT EXISTS instead of NOT IN when the sub-select could return NULLs.
NOT IN with any NULL in the list returns zero rows -- a silent, surprising result.

Sub-Selects in FROM: Inline Views

A sub-select in the FROM clause acts as a temporary table you can filter or join against -- useful for applying a WHERE to an aggregate or window result.

```
-- Top-spending customer per state
SELECT outer_q.state,
       outer_q.first_name,
       outer_q.last_name,
       outer_q.total_spent
FROM (
  SELECT s.state,
         c.first_name, c.last_name,
         ROUND(SUM(f.total), 0) AS total_spent,
         RANK() OVER (
           PARTITION BY s.state
           ORDER BY SUM(f.total) DESC
         ) AS rnk
  FROM   samples.retail.fact_sales f
  JOIN   samples.retail.dim_customer c
        ON f.customer_key = c.customer_key
  JOIN   samples.retail.dim_store s
        ON f.store_key = s.store_key
  GROUP BY s.state, c.first_name, c.last_name
) outer_q
WHERE  outer_q.rnk = 1
ORDER BY outer_q.state;
```

Sub-selects vs CTEs

When to use a sub-select

One-off filter or transform used once.

Short logic (under ~8 lines).

No need to reference it again later.

When to use a CTE instead

You reference the same result more than once.

The logic is complex enough to name.

You want to test each step in isolation.

(See next section.)

Alias required

Sub-selects in FROM must have an alias.

outer_q above is the alias. Without it, the query will return a syntax error.

CTEs: Breaking Complex Queries into Named Steps

A CTE (Common Table Expression) lets you name an intermediate result and reference it later — like creating a temporary view inside your query.

```
-- CTE: name a step, then use it
WITH store_revenue AS (
  SELECT s.store_name,
         s.state,
         COUNT(*)           AS transactions,
         ROUND(SUM(f.total),0) AS revenue
  FROM   samples.retail.fact_sales f
  JOIN   samples.retail.dim_store s
        ON f.store_key = s.store_key
  GROUP BY s.store_name, s.state
)
SELECT store_name,
       state,
       transactions,
       revenue,
       RANK() OVER (PARTITION BY state
                    ORDER BY revenue DESC) AS rank_in_state
FROM   store_revenue
ORDER BY state, rank_in_state
LIMIT 8;
```

Readability

Name your steps. `store_revenue` is clearer than a nested subquery 5 levels deep.

Reusability

Reference the same CTE multiple times in one query — no copy-paste.

Debugging

Test each CTE in isolation before wiring them together.

Multiple CTEs

Chain them with commas: `WITH a AS (...), b AS (...)`
`SELECT ...`

Formatting & Readability Habits

❌ Hard to read

```
select f.transaction_id,s.store_name,p.name,f.total from
samples.retail.fact_sales f join samples.retail.dim_store s on
f.store_key=s.store_key join samples.retail.dim_product p on
f.product_key=p.product_key where f.total>1000 order by f.total desc
limit 5
```

✅ Readable

```
SELECT f.transaction_id,
       s.store_name,
       p.name          AS product,
       f.total         AS sale_total
FROM   samples.retail.fact_sales   f
JOIN   samples.retail.dim_store   s
       ON f.store_key = s.store_key
JOIN   samples.retail.dim_product p
       ON f.product_key = p.product_key
WHERE  f.total > 1000
ORDER BY f.total DESC
LIMIT 5;
```

- UPPERCASE keywords — SELECT, FROM, WHERE, JOIN stand out from your column names
- Align column lists — line up commas, AS, and ON at the same indent level
- One clause per line — each SELECT column, JOIN, and WHERE condition on its own line
- Use comments — -- Explain why, not what. The SQL already says what.
- Meaningful aliases — s for dim_store, p for dim_product — not a, b, c

5-Point Efficiency Checklist for Every Query

1

Select only the columns you need

Skip SELECT *. Yellowbrick is columnar — every extra column costs I/O.

2

Filter early and filter raw

Put WHERE conditions on raw column values, not expressions. Avoid YEAR(col) = 2026; use col >= '2026-01-01' instead.

3

Always LIMIT when exploring

Before you know the result size, add LIMIT 100. It costs nothing and prevents surprise 10M-row returns.

4

Join on the right columns

Join on the keys the table was designed for (the _key columns). Joining on expressions or concatenations forces a full shuffle.

5

Use CTEs to stay readable

Name your intermediate steps. A query you can read in 6 months is a query you can fix in 6 months.

Questions?

docs.yellowbrick.com · support.yellowbrick.com · michael.suarez@yellowbrick.com